

Module 4 - Test Driven Development and Logging Framework

Introduction to Software Testing

- Software Testing is the process of evaluating a software application to ensure that it performs according to the specified requirements and is free from defects.
- It helps developers identify bugs, errors, and missing functionalities before the software is released to users.
- Testing improves the quality, reliability, performance, and security of software applications, making them more dependable in real-world environments.
- In software development, testing plays an important role throughout the Software Development Life Cycle (SDLC).
- Rather than waiting until the end of development, modern software projects encourage continuous testing during every phase.
- This approach reduces the cost of fixing defects because issues are detected much earlier. A well-tested application not only provides a better user experience but also minimizes maintenance efforts after deployment.
- Manual testing involves testers executing test cases without automation tools, whereas automation testing uses testing frameworks and scripts to validate software functionality.

Types of Software Testing

Software testing is broadly classified into several types based on the purpose of testing.

- **Unit Testing:** Tests individual methods or classes independently.
- **Integration Testing:** Tests the interaction between different modules.
- **System Testing:** Validates the complete application.
- **Acceptance Testing:** Confirms that the application satisfies customer requirements.
- **Performance Testing:** Measures speed, scalability, and stability.
- **Security Testing:** Identifies vulnerabilities and security risks.

Test Driven Development (TDD)

- Test Driven Development (TDD) is a software development methodology in which developers write automated test cases before writing the actual implementation code.
- Unlike the **traditional development approach, where coding is completed first and testing is performed later, TDD begins by defining the expected behaviour of the application through test cases.**
- Initially, these test cases fail because the functionality has not yet been implemented. Developers then write only the required code to make the test pass, followed by improving the code without changing its functionality.
- The primary objective of TDD is to produce clean, maintainable, and reliable software.
- Since every feature is supported by automated tests, developers can modify existing code confidently without worrying about introducing new defects.
- One of the major benefits of TDD is the **early detection of defects.**
- Since testing is integrated into the development process, bugs are identified immediately after implementation rather than during the final testing phase. This significantly reduces debugging time and improves overall software quality.
- TDD is widely used in Agile software development because it supports continuous integration and frequent code changes.

TDD Workflow (Red-Green-Refactor)

Test Driven Development follows a simple and repetitive cycle known as the **Red-Green-Refactor** cycle.

This workflow ensures that every new feature is first validated through a test case before implementation.

By following these three phases, developers can write clean, reliable, and maintainable code while minimizing the chances of introducing defects.

The Red-Green-Refactor cycle encourages developers to write only the necessary code required to satisfy the current requirement. Once the functionality is verified through automated tests, the code is improved without changing its behavior. This continuous cycle results in better software quality and easier maintenance.

Red Phase

In this stage, the developer writes a test case based on the expected functionality of the application. Since the corresponding implementation has not yet been written, the test fails. The failed test confirms that the test case is working correctly and that the required functionality is still missing.

Green Phase

The developer writes the minimum amount of code required to make the failed test pass. The objective is not to develop a perfect solution immediately but to satisfy the test case using the simplest possible implementation. Once the test passes successfully, the feature is considered functionally complete.

Refactor Phase

In this phase, the developer improves the internal structure of the code without changing its external behaviour. Code duplication is removed, variable names are improved, and unnecessary complexity is reduced. After every modification, all test cases are executed again to ensure that the functionality remains unchanged.

Advantages of the Red-Green-Refactor Cycle

- Encourages incremental software development.
- Improves code quality and readability.
- Detects defects at an early stage.
- Reduces debugging and maintenance efforts.
- Ensures every feature is supported by automated tests.

Example

Suppose a developer wants to implement a method that checks whether a number is even.

Step 1: Write the Test (Red)

```
@Test
void testEvenNumber() {
    assertTrue(NumberUtil.isEven(10));
}
```

The test fails because the `isEven()` method does not exist.

Step 2: Write the Code (Green)

```
class NumberUtil {
    static boolean isEven(int n) {
        return n % 2 == 0;
    }
}
```

Now the test passes successfully.

Step 3: Refactor

The code is reviewed and improved if necessary. Since the implementation is already simple and efficient, no further changes are required.

Unit Testing

- Unit Testing is a software testing technique in which the smallest individual components of an application, such as methods or classes, are tested independently to verify that they work correctly.
- Each unit is tested in isolation without depending on other parts of the application.
- The primary objective of unit testing is to ensure that every component performs its intended function accurately before it is integrated with other modules.
- Automated testing frameworks such as **JUnit** make it easier to create and execute unit tests repeatedly. Since these tests can be run whenever code changes are made, developers can quickly identify defects and fix them before they affect other parts of the application.

Characteristics of Unit Testing

- Tests individual methods or classes.
- Performed during the development phase.
- Usually automated using testing frameworks.
- Executes quickly.
- Helps detect defects at an early stage.

Advantages of Unit Testing

- Improves software quality.
- Makes debugging easier.
- Reduces development cost.
- Encourages modular programming.
- Supports continuous integration.

Unit Testing vs Integration Testing

Unit Testing	Integration Testing
Tests a single unit of code.	Tests interaction between multiple modules.
Performed by developers.	Performed after unit testing.
Fast execution.	Comparatively slower.
Easy to locate defects.	More difficult to identify the exact source of defects.

Example

```
public class Calculator {  
    public int multiply(int a, int b) {  
        return a * b;  
    }  
}  
  
import static org.junit.jupiter.api.Assertions.*;  
import org.junit.jupiter.api.Test;  
  
class CalculatorTest {
```

```
@Test

void testMultiply() {

    Calculator c = new Calculator();

    assertEquals(20, c.multiply(4,5));

}

}
```

In the above example, the multiply() method is tested independently using JUnit. The assertEquals() statement compares the expected result (20) with the actual result returned by the method. If both values are equal, the test passes; otherwise, JUnit reports a failure, helping the developer identify the issue quickly.

JUnit 5

Introduction

JUnit 5 is one of the most popular testing frameworks for Java applications. It is primarily used to write and execute unit test cases efficiently. JUnit enables developers to verify whether individual methods or classes behave as expected by comparing the actual output with the expected result. It simplifies the testing process through annotations, assertions, and automated execution of test cases. JUnit 5 is the latest version of the JUnit framework and consists of three major components:

- **JUnit Platform,**
- **JUnit Jupiter,**
- **JUnit Vintage.**

JUnit Platform provides the foundation for launching testing frameworks, JUnit Jupiter contains the new programming model and extension model for writing tests, while JUnit Vintage allows execution of older JUnit 3 and JUnit 4 test cases.

Features of JUnit 5

- Easy to write and execute test cases.
- Supports annotations for test lifecycle management.
- Provides various assertion methods.
- Supports parameterized testing.
- Allows exception and timeout testing.
- Integrates easily with Maven, Gradle, Spring Boot, and IDEs like IntelliJ IDEA and Eclipse.

Common JUnit Annotations

Annotation	Purpose
@Test	Marks a method as a test case.
@BeforeEach	Executes before every test method.
@AfterEach	Executes after every test method.
@BeforeAll	Executes once before all test methods.
@AfterAll	Executes once after all test methods.
@DisplayName	Provides a custom name for the test.

@Disabled	Skips execution of a test method.
-----------	-----------------------------------

Common Assertions

JUnit provides several assertion methods to validate expected results.

Assertion	Description
assertEquals()	Checks whether two values are equal.
assertTrue()	Verifies whether a condition is true.
assertFalse()	Verifies whether a condition is false.
assertNull()	Checks whether an object is null.
assertNotNull()	Checks whether an object is not null.
assertThrows()	Verifies that an exception is thrown.

Example 1: Calculator Program using JUnit 5

Calculator.java

```
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }

    public int divide(int a, int b) {
        return a / b;
    }

}
```

CalculatorTest.java

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class CalculatorTest {

    Calculator calculator = new Calculator();

    @Test
    void testAddition() {
        assertEquals(10, calculator.add(6,4));
    }

    @Test
    void testSubtraction() {
        assertEquals(5, calculator.subtract(10,5));
    }

}
```

```

}

@Test
void testMultiplication() {
    assertEquals(20, calculator.multiply(4,5));
}

@Test
void testDivision() {
    assertEquals(5, calculator.divide(20,4));
}
}

```

Output

Tests run: 4

Addition Test ✓

Subtraction Test ✓

Multiplication Test ✓

Division Test ✓

The screenshot shows an IDE with two tabs: 'calculator.java' and 'calculatorTest.java'. The 'calculatorTest.java' tab is active, displaying the following code:

```

1 package test;
2 import static org.junit.jupiter.api.Assertions.*;
3 import main.calculator;
4 import org.junit.jupiter.api.Test;
5
6 public class calculatorTest {
7     main.calculator calculator = new calculator(); 4 usages
8     @Test
9     void testAddition() {
10         assertEquals( expected: 10, calculator.add( a: 6, b: 4));
11     }
12
13     @Test
14     void testSubtraction() {
15         assertEquals( expected: 5, calculator.subtract( a: 10, b: 5));
16     }
17
18     @Test
19     void testMultiplication() {
20         assertEquals( expected: 20, calculator.multiply( a: 4, b: 5));
21     }
22
23     @Test
24     void testDivision() {
25         assertEquals( expected: 5, calculator.divide( a: 20, b: 4));
26     }
27 }

```

On the right side, the 'Run' tab is open, showing the execution results for 'calculatorTest (test)'. The results are:

- ✓ testMultiplication()
- ✓ testAddition()
- ✓ testDivision()
- ✓ testSubtraction()

Below the test results, it states: '✓ Tests passed: 4 of 4 tests - 62 ms'. The command used for execution is 'C:\Users\yagnatharun\.jdk\openjdk-23\bin\java.exe ..'. The final message is 'Process finished with exit code 0'.

Explanation

The Calculator class contains four basic arithmetic operations: addition, subtraction, multiplication, and division. The CalculatorTest class creates an object of the Calculator class and verifies each method using the assertEquals() assertion. If the expected value matches the actual result returned by the method, the test passes successfully. Otherwise, JUnit reports a failure indicating the incorrect functionality.

Example 2: Using @BeforeEach and @AfterEach

Student.java

```

public class Student {

    private String name;

```

```
public Student(String name) {
    this.name = name;
}

public String getName() {
    return name;
}

}
```

StudentTest.java

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.*;

public class StudentTest {

    Student student;

    @BeforeEach
    void setup() {
        student = new Student("Yagna");
        System.out.println("Object Created");
    }

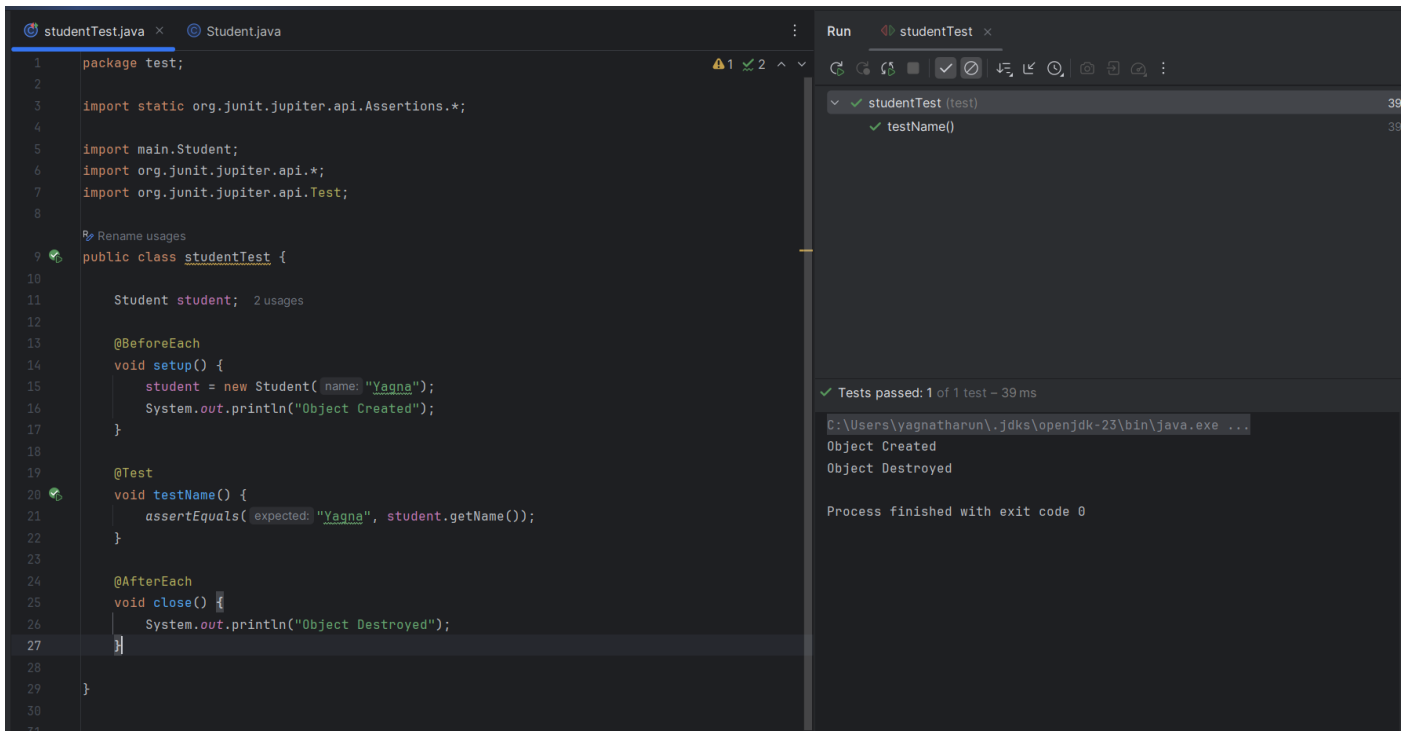
    @Test
    void testName() {
        assertEquals("Yagna", student.getName());
    }

    @AfterEach
    void close() {
        System.out.println("Object Destroyed");
    }

}
```

Output

Object Created
Object Destroyed



Explanation

The `@BeforeEach` annotation executes before every test case and is commonly used to initialize objects or resources. The `@AfterEach` annotation executes after every test case and is mainly used for cleanup operations. This ensures that each test starts with a fresh environment and avoids interference between test cases.

Example 3: Exception Testing

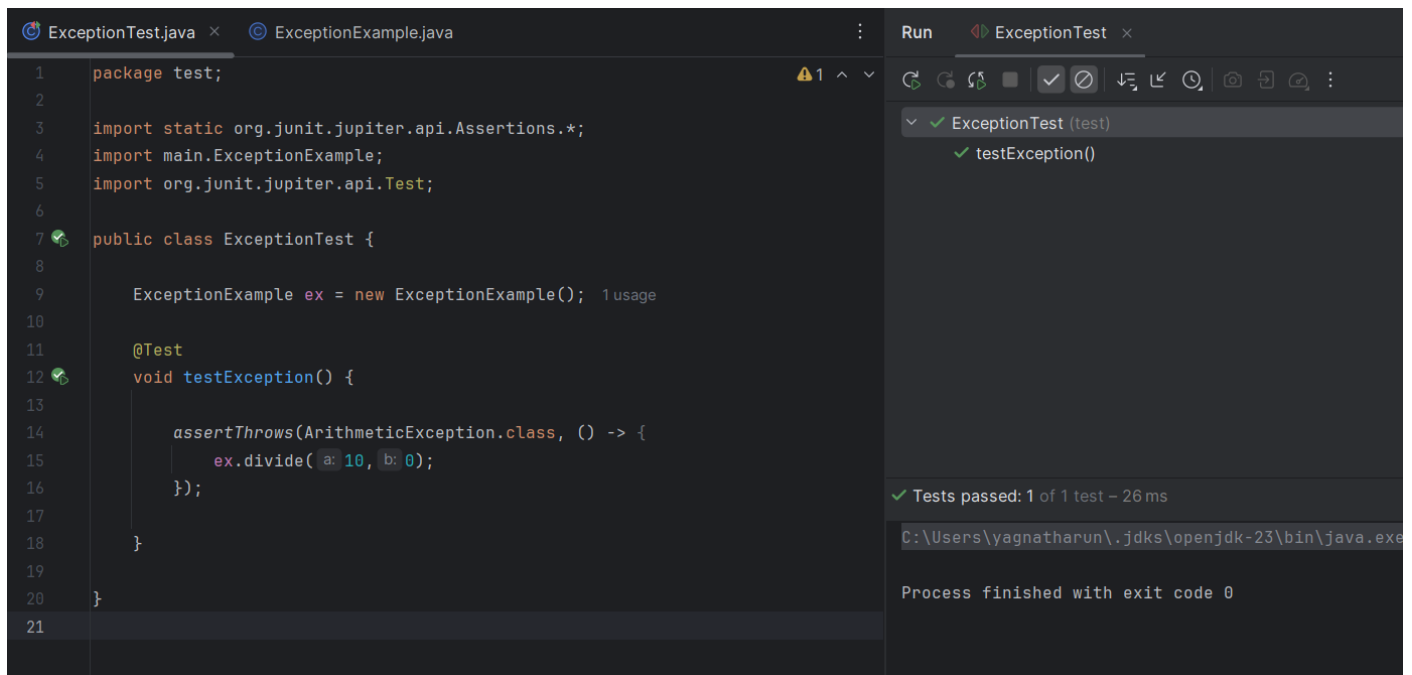
ExceptionExample.java

```
public class ExceptionExample {  
  
    public int divide(int a, int b) {  
        return a / b;  
    }  
  
}
```

ExceptionTest.java

```
import static org.junit.jupiter.api.Assertions.*;  
import org.junit.jupiter.api.Test;  
  
public class ExceptionTest {  
  
    ExceptionExample ex = new ExceptionExample();  
  
    @Test  
    void testException() {  
  
        assertThrows(ArithmeticException.class, () -> {  
            ex.divide(10,0);  
        });  
  
    }  
  
}
```


Output



The screenshot shows an IDE with two tabs: 'ExceptionTest.java' and 'ExceptionExample.java'. The 'ExceptionTest.java' tab is active, displaying the following code:

```
1 package test;
2
3 import static org.junit.jupiter.api.Assertions.*;
4 import main.ExceptionExample;
5 import org.junit.jupiter.api.Test;
6
7 public class ExceptionTest {
8
9     ExceptionExample ex = new ExceptionExample(); // 1 usage
10
11     @Test
12     void testException() {
13
14         assertThrows(ArithmeticException.class, () -> {
15             ex.divide(a: 10, b: 0);
16         });
17
18     }
19
20 }
21
```

On the right side, the 'Run' tab is active, showing the test results for 'ExceptionTest (test)'. The test 'testException()' is marked as passed. Below the test results, it states 'Tests passed: 1 of 1 test - 26 ms' and 'Process finished with exit code 0'.

Explanation

The `assertThrows()` method verifies that the specified exception is thrown during execution. In this example, dividing a number by zero generates an `ArithmeticException`, and JUnit confirms that the expected exception occurs. This feature is useful for validating error-handling scenarios.

Arrange-Act-Assert (AAA) Pattern

The **Arrange-Act-Assert (AAA) Pattern** is one of the most commonly used structures for writing unit test cases.

It provides a clear and organized way to write tests by dividing each test into three logical sections.

Following this pattern makes test cases easier to understand, maintain, and debug.

First, the required objects and input values are prepared. Next, the method under test is executed. Finally, the result is verified using assertions.

This simple structure is widely used in JUnit, NUnit, xUnit, and many other testing frameworks.

Developers prefer the AAA pattern because it reduces confusion and ensures that every test follows a consistent structure. Whether the project contains ten test cases or thousands, using AAA makes the test suite easier to maintain.

Phases of the AAA Pattern

1. Arrange

The Arrange phase is used to prepare all the necessary objects, variables, and input data required for testing. Any initialization, object creation, or setup required before executing the test is performed in this phase.

Example:

```
Calculator calculator = new Calculator();
int a = 10;
int b = 20;
```

2. Act

In the Act phase, the actual method being tested is executed. Only one action should generally be performed in this phase so that the purpose of the test remains clear.

Example:

```
int result = calculator.add(a, b);
```

3. Assert

The Assert phase verifies whether the actual output matches the expected result. JUnit provides various assertion methods for validating the output.

Example:

```
assertEquals(30, result);
```

Complete Example using AAA Pattern

Calculator.java

```
public class Calculator {  
  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
}
```

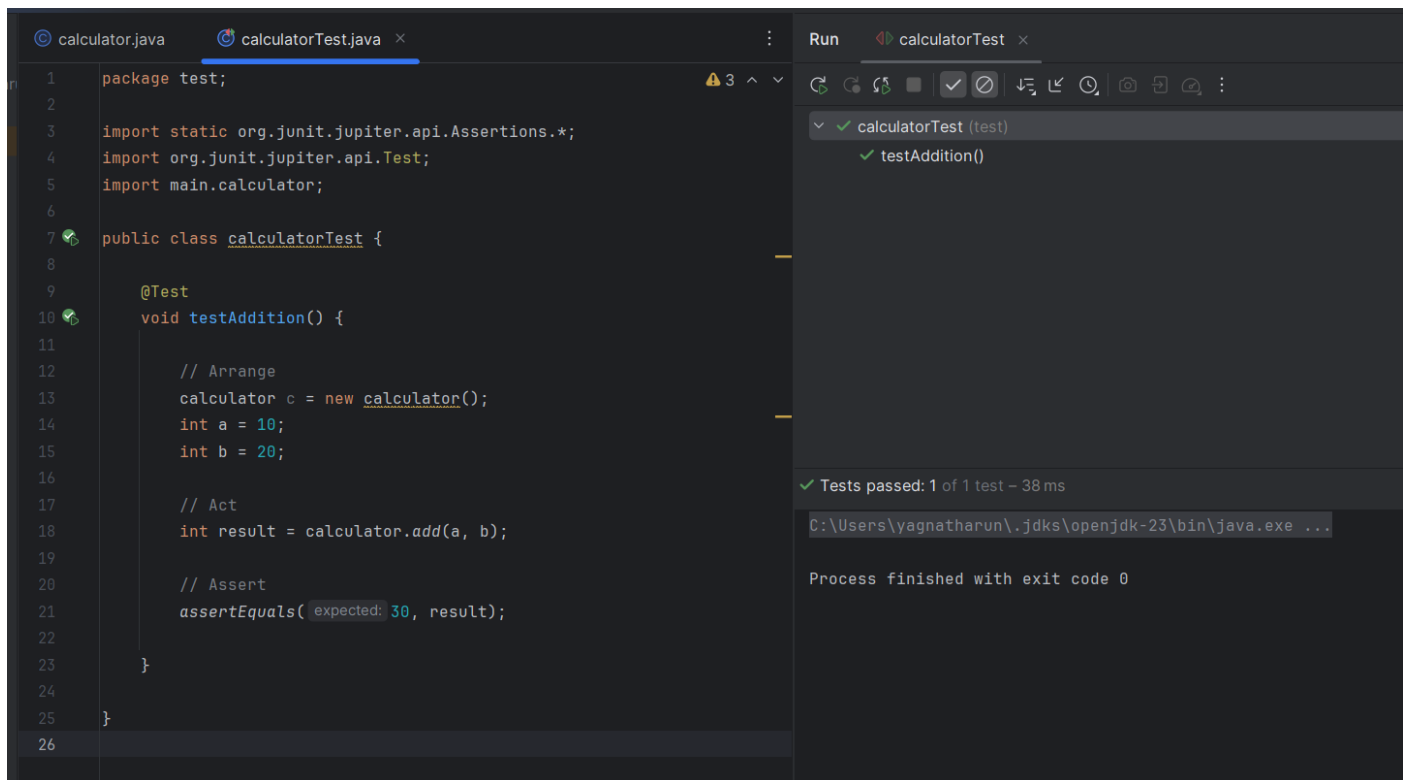
CalculatorTest.java

```
import static org.junit.jupiter.api.Assertions.*;  
import org.junit.jupiter.api.Test;
```

```
public class CalculatorTest {  
  
    @Test  
    void testAddition() {  
  
        // Arrange  
        Calculator calculator = new Calculator();  
        int a = 10;  
        int b = 20;  
  
        // Act  
        int result = calculator.add(a, b);  
  
        // Assert  
        assertEquals(30, result);  
  
    }  
  
}
```

Output

Test Passed



The screenshot shows an IDE with two tabs: `calculator.java` and `calculatorTest.java`. The `calculatorTest.java` file contains the following code:

```
1 package test;
2
3 import static org.junit.jupiter.api.Assertions.*;
4 import org.junit.jupiter.api.Test;
5 import main.calculator;
6
7 public class calculatorTest {
8
9     @Test
10    void testAddition() {
11
12        // Arrange
13        calculator c = new calculator();
14        int a = 10;
15        int b = 20;
16
17        // Act
18        int result = calculator.add(a, b);
19
20        // Assert
21        assertEquals( expected: 30, result);
22    }
23 }
24
25
26
```

On the right, the `Run` tab shows the execution of `calculatorTest`. It displays a tree view with `calculatorTest (test)` and `testAddition()`, both marked with green checkmarks. Below this, it states: `Tests passed: 1 of 1 test - 38 ms`. The command line shows `C:\Users\yagnatharun\.jdk\openjdk-23\bin\java.exe ...` and the final message is `Process finished with exit code 0`.

Explanation

In this example, the Calculator object and input values are created during the Arrange phase. The `add()` method is executed during the Act phase, and finally, the Assert phase verifies that the returned value is equal to 30. Since the expected and actual values match, the test passes successfully.

Advantages of the AAA Pattern

- Makes test cases easier to read.
- Improves code organization.
- Simplifies debugging.
- Encourages writing one test for one functionality.
- Provides consistency across the entire test suite.

Advanced JUnit Features

JUnit 5 provides several advanced features that make testing more flexible and efficient. These features help developers execute the same test with multiple inputs, organize related test cases, verify exceptions, measure execution time, and control the order of test execution.

They improve the quality of automated testing while reducing duplicate code.

Some of the most commonly used advanced features are Parameterized Tests, Test Suites, Exception Testing, Timeout Testing, and Test Execution Order.

Parameterized Tests

A **Parameterized Test** allows the same test case to run multiple times with different input values. Instead of writing separate test methods for different test data, developers can use parameterized tests to improve code reusability.

Example

NumberUtil.java

```

public class NumberUtil {

    public boolean isEven(int number) {
        return number % 2 == 0;
    }

}

```

NumberUtilTest.java

```

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

public class NumberUtilTest {

    NumberUtil numberUtil = new NumberUtil();

    @ParameterizedTest
    @ValueSource(ints = {2,4,6,8,10})
    void testEvenNumbers(int number) {

        assertTrue(numberUtil.isEven(number));

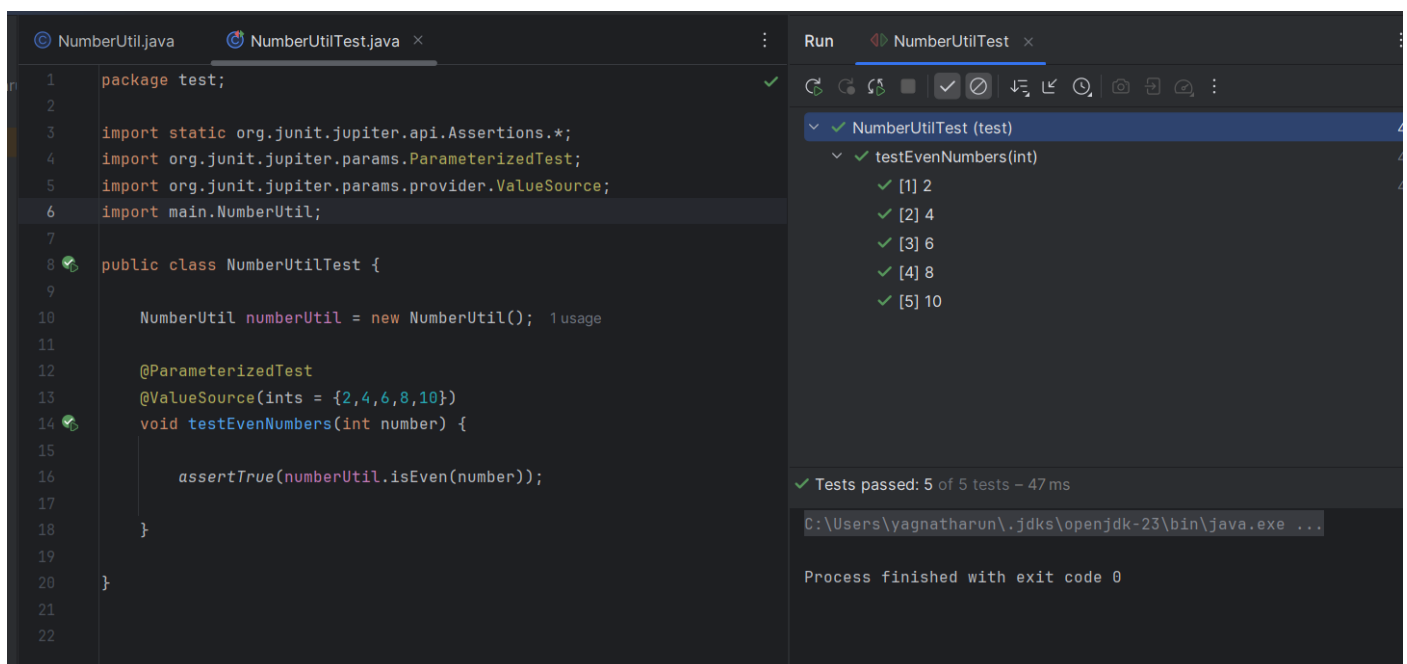
    }

}

```

Output

Input : 2 ✓
 Input : 4 ✓
 Input : 6 ✓
 Input : 8 ✓
 Input : 10 ✓



The screenshot shows an IDE with two panes. The left pane displays the source code of `NumberUtilTest.java`, which includes imports for JUnit 5 and JUnit 5 Parameterized Test, and a test class `NumberUtilTest` with a parameterized test `testEvenNumbers`. The right pane shows the Run console output, indicating that all 5 tests passed successfully.

Run Console Output:

```

Run NumberUtilTest
✓ NumberUtilTest (test)
  ✓ testEvenNumbers(int)
    ✓ [1] 2
    ✓ [2] 4
    ✓ [3] 6
    ✓ [4] 8
    ✓ [5] 10

✓ Tests passed: 5 of 5 tests – 47 ms
C:\Users\yagnatharun\.jdk\openjdk-23\bin\java.exe ...
Process finished with exit code 0

```

Explanation

The `@ParameterizedTest` annotation executes the same test method multiple times. The `@ValueSource` annotation provides different input values, allowing a single test method to validate multiple scenarios.

Test Suites

A **Test Suite** is used to group multiple test classes together so they can be executed simultaneously. This is useful for large projects where different modules have separate test classes.

Example

```
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;
```

```
@Suite
@SelectClasses({
    CalculatorTest.class,
    StudentTest.class,
    NumberUtilTest.class
})
public class AllTests {

}
```

Output

```
Running CalculatorTest...
Running StudentTest...
Running NumberUtilTest...
All Tests Passed
```

Timeout Testing

Sometimes a method may take too much time to execute because of inefficient logic or infinite loops. JUnit provides the `assertTimeout()` method to ensure that a method completes within a specified duration.

Example

```
import static org.junit.jupiter.api.Assertions.*;
import java.time.Duration;
import org.junit.jupiter.api.Test;

public class TimeoutTest {

    @Test
    void timeoutExample() {

        assertTimeout(Duration.ofSeconds(2), () -> {

            Thread.sleep(1000);

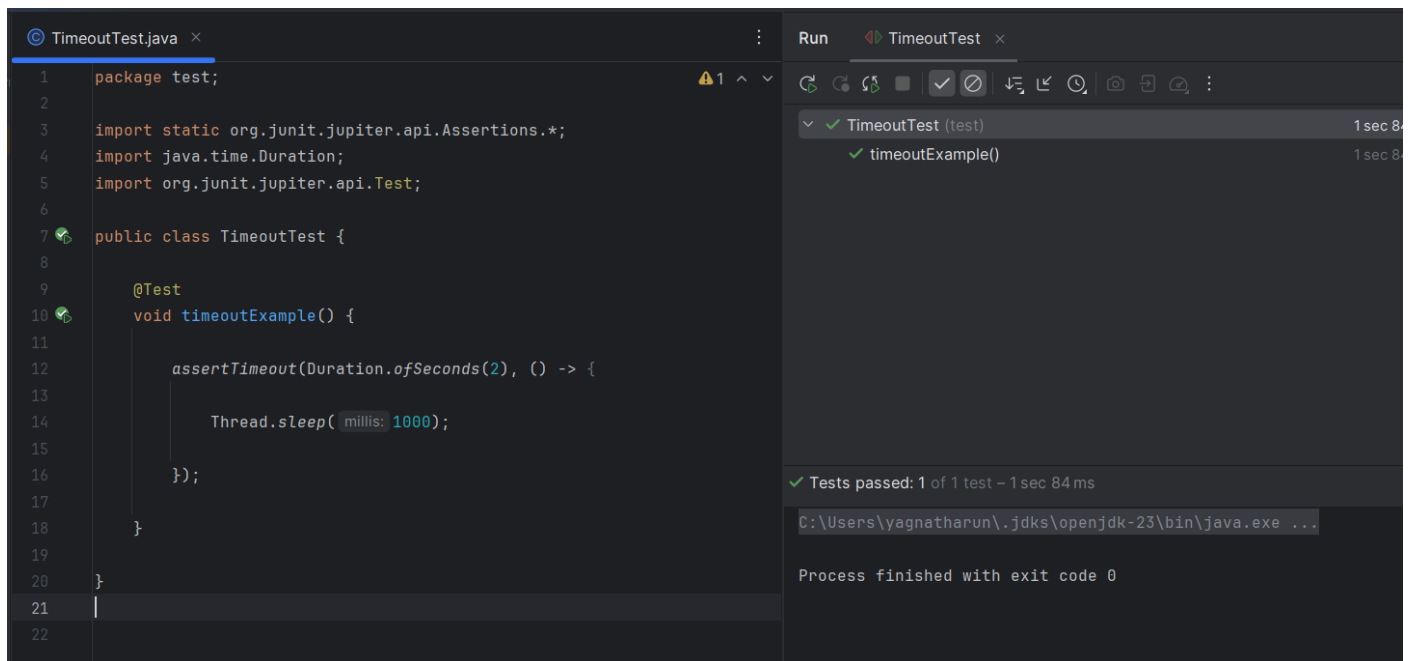
        });

    }

}
```

Output

Execution Time : 1 Second



Test Execution Order

JUnit normally executes test methods in a random order. However, developers can specify the execution order whenever required.

Example

```
import org.junit.jupiter.api.*;
```

```
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
```

```
public class OrderTest {
```

```
    @Test
```

```
    @Order(1)
```

```
    void login() {
```

```
        System.out.println("Login");
```

```
    }
```

```
    @Test
```

```
    @Order(2)
```

```
    void search() {
```

```
        System.out.println("Search");
```

```
    }
```

```
    @Test
```

```
    @Order(3)
```

```
    void logout() {
```

```
        System.out.println("Logout");
```

```
    }
```

```
}
```

Output

Login

Search

Logout

```
1 import org.junit.jupiter.api.*;
2
3 @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
4 public class OrderTest {
5
6     @Test
7     @Order(1)
8     void login() {
9         System.out.println("Login");
10    }
11
12    @Test
13    @Order(2)
14    void search() {
15        System.out.println("Search");
16    }
17
18    @Test
19    @Order(3)
20    void logout() {
21        System.out.println("Logout");
22    }
23
24 }
25
26
```

Run OrderTest

- ✓ OrderTest
- ✓ login()
- ✓ search()
- ✓ logout()

✓ Tests passed: 3 of 3 tests – 25 ms

C:\Users\yagnatharun\.jdk\openjdk-23\bin\java.exe ...

Login
Search
Logout

Process finished with exit code 0

Mockito

Mockito is one of the most popular Java testing frameworks used for creating **mock objects** during unit testing. It allows developers to test a class independently without depending on external resources such as databases, web services, file systems, or other classes.

In real-world applications, many classes depend on other classes to perform their operations. Testing such classes becomes difficult because those dependencies may require database connections, network access, or complex configurations. Mockito solves this problem by replacing real objects with mock objects that simulate their behavior. This makes unit testing faster, simpler, and more reliable.

Mockito is commonly used together with JUnit. While JUnit provides the framework for writing and executing test cases, Mockito provides the ability to create mock objects and define their behavior.

- Creates mock objects easily.
- Eliminates dependency on databases or APIs during testing.
- Supports stubbing methods using `when()` and `thenReturn()`.
- Verifies whether methods are called using `verify()`.
- Integrates seamlessly with JUnit 5.
- Makes unit tests faster and more reliable.

Mock Object vs Real Object

Real Object	Mock Object
Contains actual implementation.	Simulates the behavior of a real object.
May access databases or APIs.	Does not access external resources.
Slower during testing.	Faster execution.
Used in production.	Used only for testing.

Example 1: Creating a Mock Object

EmployeeRepository.java

```
public interface EmployeeRepository {  
  
    String getEmployeeName();  
  
}
```

EmployeeService.java

```
public class EmployeeService {  
  
    private EmployeeRepository repository;  
  
    public EmployeeService(EmployeeRepository repository) {  
        this.repository = repository;  
    }  
  
    public String fetchEmployee() {  
        return repository.getEmployeeName();  
    }  
  
}
```

EmployeeServiceTest.java

```
import static org.junit.jupiter.api.Assertions.*;  
import static org.mockito.Mockito.*;  
  
import org.junit.jupiter.api.Test;  
  
public class EmployeeServiceTest {  
  
    @Test  
    void testEmployeeName() {  
  
        EmployeeRepository repository = mock(EmployeeRepository.class);  
  
        when(repository.getEmployeeName()).thenReturn("Yagna");  
  
        EmployeeService service = new EmployeeService(repository);  
  
        assertEquals("Yagna", service.fetchEmployee());  
  
    }  
  
}
```

Output

Test Passed

BUILD SUCCESSFUL

Explanation

The mock() method creates a fake implementation of the EmployeeRepository interface. The when() method specifies what value should be returned whenever the getEmployeeName() method is called. During testing, the EmployeeService object communicates only with the mock object instead of a real database. Since the expected value matches the actual value, the test passes successfully.

Stubbing using when() and thenReturn()

Stubbing is the process of defining how a mock object should behave when one of its methods is called. Mockito uses the when() and thenReturn() methods for this purpose.

Example

```
when(repository.getEmployeeName()).thenReturn("John");
```

The above statement tells Mockito to return **"John"** whenever the getEmployeeName() method is invoked.

Verifying Method Calls

Mockito also allows developers to verify whether a particular method was called during test execution.

Example

```
import static org.mockito.Mockito.*;
```

```
import org.junit.jupiter.api.Test;
```

```
public class VerifyExample {
```

```
    @Test
```

```
    void verifyMethodCall() {
```

```
        EmployeeRepository repository = mock(EmployeeRepository.class);
```

```
        repository.getEmployeeName();
```

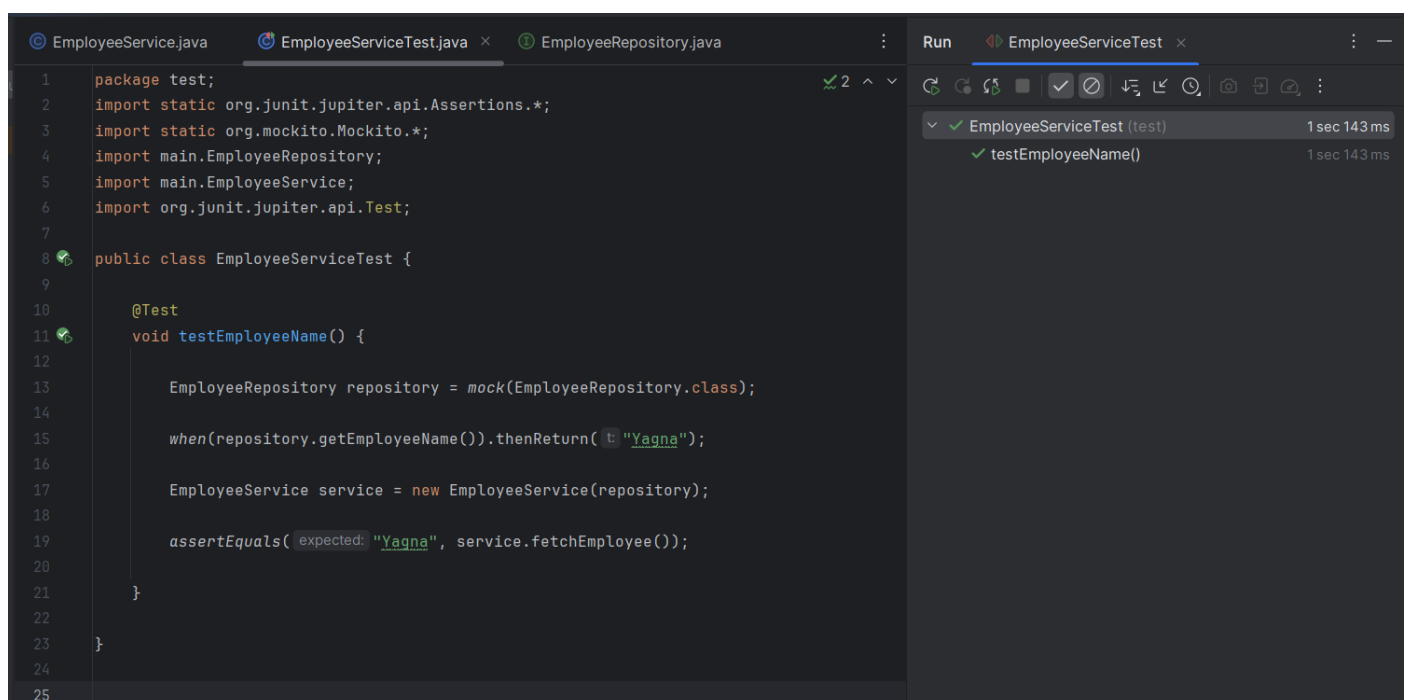
```
        verify(repository).getEmployeeName();
```

```
    }
```

```
}
```

Output

Method getEmployeeName() called successfully.



The screenshot displays an IDE with two panels. The left panel shows the source code of `EmployeeServiceTest.java`. The code defines a test class with a single test method `testEmployeeName()`. Inside this method, an `EmployeeRepository` mock is created, `when(repository.getEmployeeName()).thenReturn("Yagna");` is used to stub the `getEmployeeName()` method to return "Yagna". An `EmployeeService` instance is then created using the mock repository, and `assertEquals("Yagna", service.fetchEmployee());` is used to verify that the service returns the expected name.

```
1 package test;
2 import static org.junit.jupiter.api.Assertions.*;
3 import static org.mockito.Mockito.*;
4 import main.EmployeeRepository;
5 import main.EmployeeService;
6 import org.junit.jupiter.api.Test;
7
8 public class EmployeeServiceTest {
9
10     @Test
11     void testEmployeeName() {
12
13         EmployeeRepository repository = mock(EmployeeRepository.class);
14
15         when(repository.getEmployeeName()).thenReturn("Yagna");
16
17         EmployeeService service = new EmployeeService(repository);
18
19         assertEquals("Yagna", service.fetchEmployee());
20     }
21 }
22
23
24
25
```

The right panel shows the 'Run' output for `EmployeeServiceTest`. It indicates that the test passed successfully, with the following results:

Test Case	Duration
EmployeeServiceTest (test)	1 sec 143 ms
testEmployeeName()	1 sec 143 ms

Explanation

The `verify()` method checks whether the specified method was invoked during execution. If the method was never called, Mockito reports a test failure. This feature is useful when testing interactions between different classes.

Mockito Annotations

Mockito provides several annotations that reduce boilerplate code and make test classes cleaner.

Annotation	Purpose
@Mock	Creates a mock object.
@InjectMocks	Injects mock objects into the class being tested.
@Spy	Creates a partial mock object.
@Captor	Captures method arguments for verification.

Example using @Mock and @InjectMocks

```
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;

public class EmployeeServiceTest {

    @Mock
    EmployeeRepository repository;

    @InjectMocks
    EmployeeService service;

    public EmployeeServiceTest() {
        MockitoAnnotations.openMocks(this);
    }

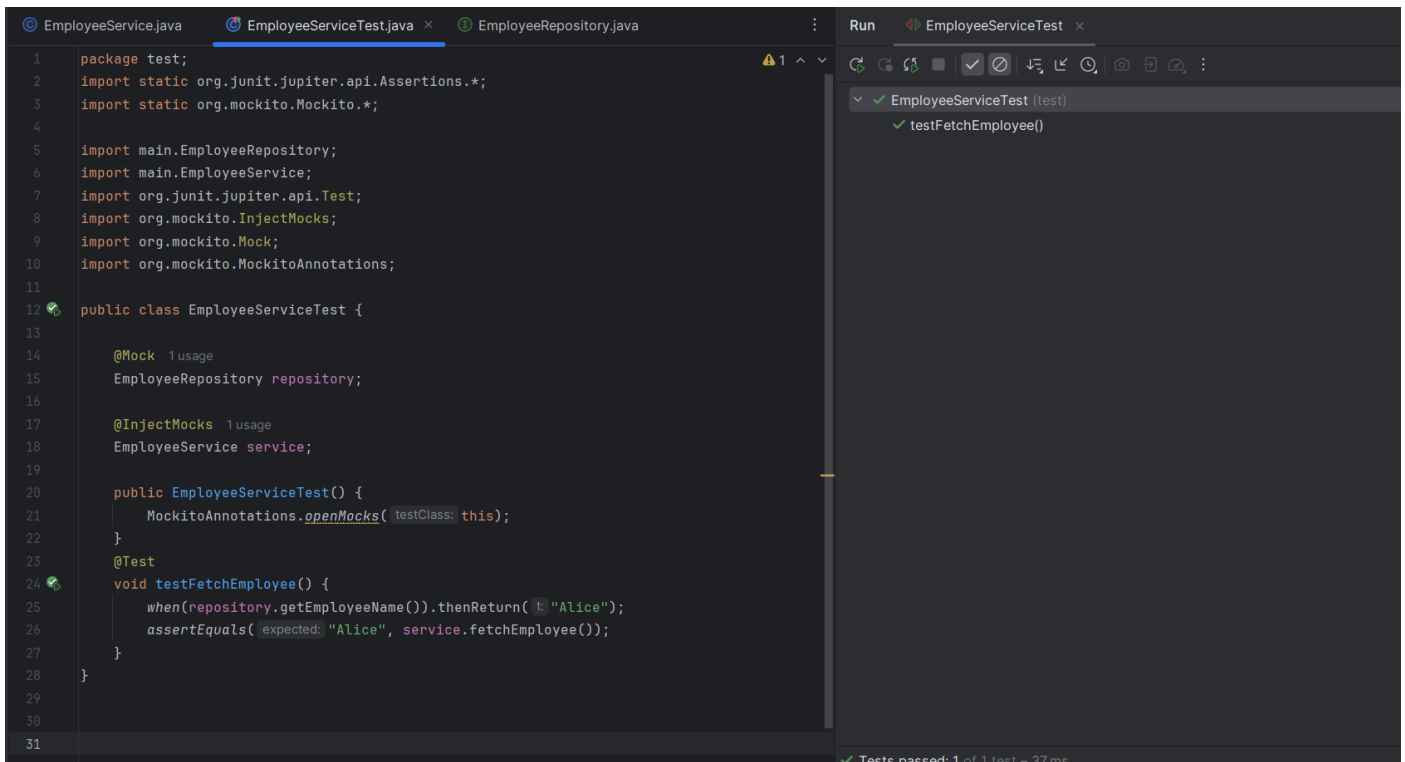
    @Test
    void testFetchEmployee() {

        when(repository.getEmployeeName()).thenReturn("Alice");

        assertEquals("Alice", service.fetchEmployee());

    }

}
```



Testing Spring Boot Applications

Spring Boot provides built-in support for testing using **JUnit 5**, **Mockito**, and the **Spring Test Framework**. These tools help developers write unit tests, integration tests, and web layer tests efficiently.

A Spring Boot application usually consists of multiple layers such as the **Controller Layer**, **Service Layer**, and **Repository Layer**. Each layer has a specific responsibility, and therefore each should be tested independently. By testing every layer separately, developers can identify defects quickly and improve application reliability.

Layers of a Spring Boot Application

A typical Spring Boot application consists of the following layers:

- **Controller Layer:** Handles HTTP requests and responses.
- **Service Layer:** Contains the business logic of the application.
- **Repository Layer:** Interacts with the database.
- **Database Layer:** Stores application data.

Example: Service Layer Testing

Student.java

```

public class Student {

    private int id;
    private String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public String getName() {
        return name;
    }
}

```

```
}
```

StudentRepository.java

```
public interface StudentRepository {
```

```
    Student findStudent();
```

```
}
```

StudentService.java

```
public class StudentService {
```

```
    private StudentRepository repository;
```

```
    public StudentService(StudentRepository repository) {  
        this.repository = repository;  
    }
```

```
    public String getStudentName() {  
        return repository.findStudent().getName();  
    }
```

```
}
```

StudentServiceTest.java

```
import static org.junit.jupiter.api.Assertions.*;  
import static org.mockito.Mockito.*;
```

```
import org.junit.jupiter.api.Test;
```

```
public class StudentServiceTest {
```

```
    @Test  
    void testStudentName() {
```

```
        StudentRepository repository = mock(StudentRepository.class);
```

```
        when(repository.findStudent())  
            .thenReturn(new Student(101, "Yagna"));
```

```
        StudentService service = new StudentService(repository);
```

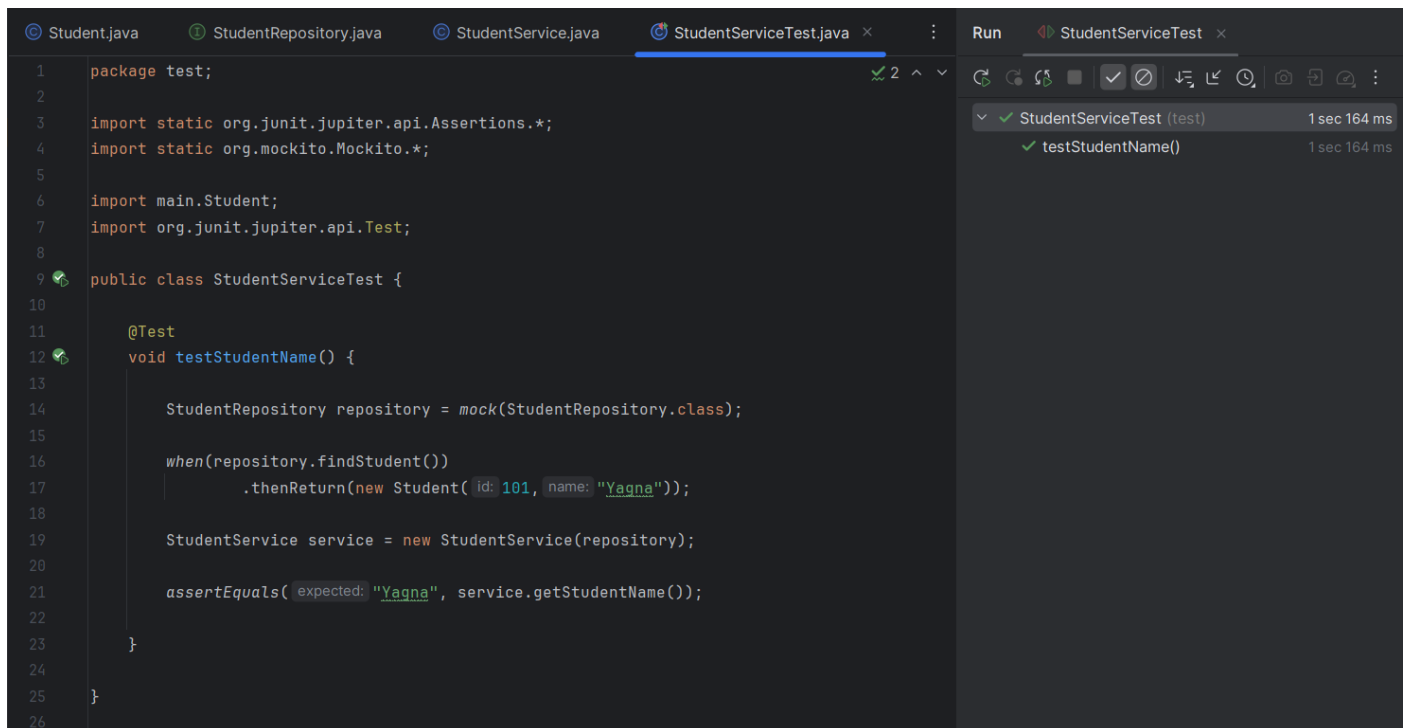
```
        assertEquals("Yagna", service.getStudentName());
```

```
    }
```

```
}
```

Output

Test Passed



```
1 package test;
2
3 import static org.junit.jupiter.api.Assertions.*;
4 import static org.mockito.Mockito.*;
5
6 import main.Student;
7 import org.junit.jupiter.api.Test;
8
9 public class StudentServiceTest {
10
11     @Test
12     void testStudentName() {
13
14         StudentRepository repository = mock(StudentRepository.class);
15
16         when(repository.findStudent())
17             .thenReturn(new Student(id: 101, name: "Yagna"));
18
19         StudentService service = new StudentService(repository);
20
21         assertEquals( expected: "Yagna", service.getStudentName());
22     }
23 }
24
25 }
26
```

Run StudentServiceTest

- ✓ StudentServiceTest (test) 1 sec 164 ms
- ✓ testStudentName() 1 sec 164 ms

Explanation

In this example, the `StudentRepository` is mocked using Mockito. Instead of connecting to an actual database, it returns a predefined `Student` object. The `StudentService` class retrieves the student's name from the mock repository, and JUnit verifies whether the returned value matches the expected result. This approach allows the service layer to be tested independently without relying on external resources.

Mocking External Dependencies

Many applications depend on external resources such as databases, REST APIs, file systems, payment gateways, and email servers. Testing applications using these real resources can be slow, expensive, and unreliable. To overcome this problem, developers use **mocking**.

Mocking is the process of **replacing an actual dependency with a fake object that behaves like the real object**. Instead of communicating with external systems, the application interacts with mock objects created using frameworks such as Mockito. This makes unit tests fast, predictable, and independent of external services.

Mocking is especially useful when external systems are unavailable or when developers want to test only the business logic of an application. By isolating dependencies, developers can verify that their code behaves correctly under different scenarios.

Common External Dependencies

- Database
- REST APIs
- Payment Gateway
- File System
- Email Service
- Cloud Storage

Example

Suppose an online shopping application calculates the total bill and sends the payment request to an external payment gateway.

Instead of connecting to the actual payment gateway during testing, Mockito creates a mock payment service that always returns **"Payment Successful"**. This allows developers to verify the billing logic without depending on the external payment system.

Automation Testing

Automation Testing is a software testing technique in which test cases are **executed automatically** using specialized tools and scripts instead of being performed manually. Automation testing helps verify software functionality quickly, accurately, and repeatedly without human intervention. It is widely used in modern software development because it saves time and increases testing efficiency.

In Agile and DevOps environments, applications are updated frequently. Manually testing every feature after each update is time-consuming and prone to human error. Automation testing solves this problem by allowing predefined test scripts to run automatically whenever changes are made to the application. This ensures that existing features continue to work correctly even after new functionality is added.

Automation testing is most beneficial for repetitive tasks, regression testing, performance testing, and large-scale applications. However, manual testing is still necessary for usability testing, exploratory testing, and scenarios requiring human judgment.

Manual Testing vs Automation Testing

Manual Testing	Automation Testing
Performed manually by testers.	Executed using automated tools.
Slower execution.	Faster execution.
Suitable for small projects.	Suitable for large projects.
More human errors.	Fewer human errors.
Less expensive initially.	Higher initial setup cost.

Popular Automation Testing Tools

Tool	Purpose
Selenium	Web Application Testing
JUnit	Unit Testing
TestNG	Java Testing
Mockito	Mock Object Creation
Cypress	Front-end Testing
Appium	Mobile Application Testing
Cucumber	Behavior Driven Development (BDD)

SLF4J (Simple Logging Facade for Java)

SLF4J (Simple Logging Facade for Java) is a logging framework that provides a simple and consistent API for generating log messages in Java applications.

Instead of depending on a specific logging implementation such as Log4j or Logback, developers write logging statements using the SLF4J API. At runtime, SLF4J forwards these log messages to the configured logging framework. This flexibility allows developers to switch logging implementations without modifying the application code.

Logging is an essential part of software development because it helps developers understand the execution flow of an application. During development, testing, and production, log messages provide valuable information about the

application's behavior, making it easier to identify errors and debug problems. Unlike `System.out.println()`, logging frameworks provide different log levels, timestamps, and configurable output destinations.

SLF4J is widely used in Spring Boot applications because it integrates seamlessly with Logback, the default logging framework in Spring Boot. Developers can easily generate logs for debugging, monitoring, and tracking application events.

Log Levels

SLF4J provides different log levels based on the importance of the message.

Log Level	Purpose
TRACE	Detailed information for debugging.
DEBUG	Information useful during development.
INFO	General application information.
WARN	Indicates a potential problem.
ERROR	Indicates serious errors or exceptions.

Example Program

Employee.java

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class Employee {

    private static final Logger logger =
        LoggerFactory.getLogger(Employee.class);

    public static void main(String[] args) {

        logger.info("Application Started");

        int salary = 50000;

        logger.debug("Employee Salary : {}", salary);

        if(salary > 40000)
            logger.warn("High Salary");

        try {
```

```

        int result = 10 / 0;

    }

    catch(Exception e){

        logger.error("Exception Occurred : {}", e.getMessage());

    }

    logger.info("Application Ended");

}

}

```

Output

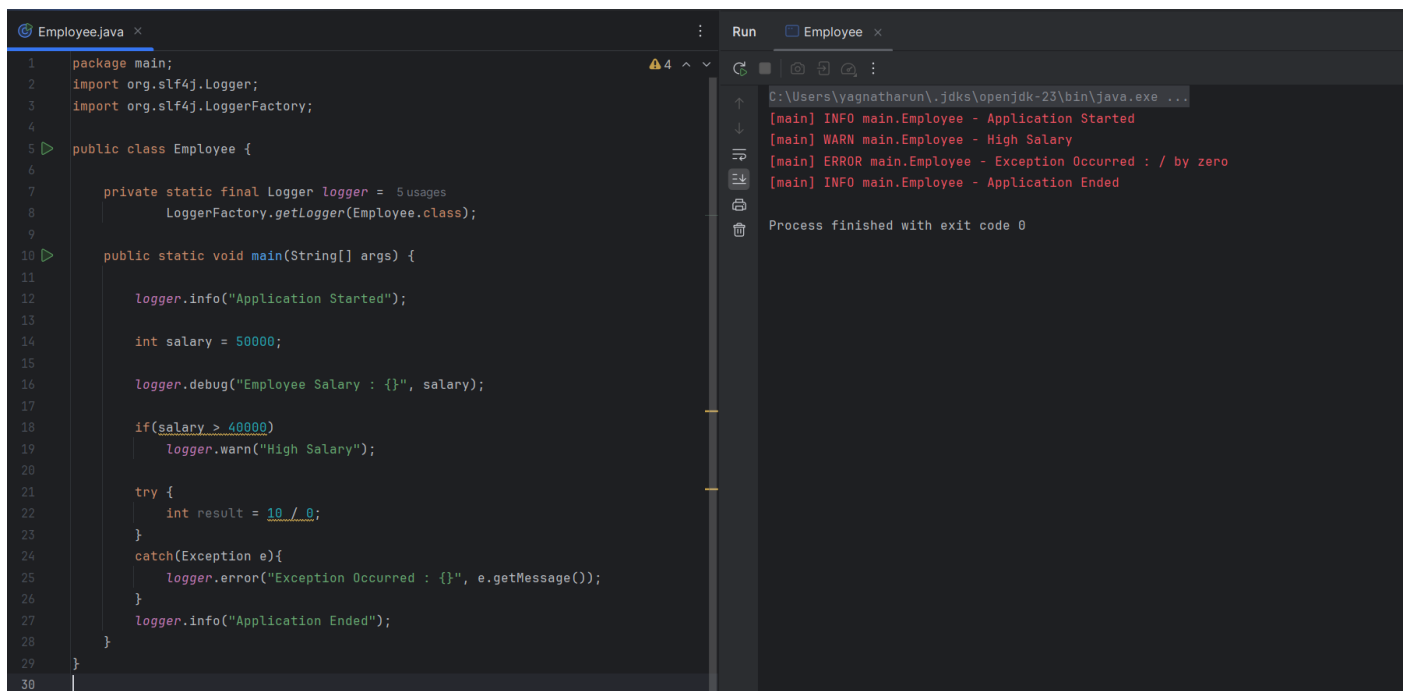
INFO - Application Started

DEBUG - Employee Salary : 50000

WARN - High Salary

ERROR - Exception Occurred : / by zero

INFO - Application Ended



The screenshot shows an IDE with a Java file named 'Employee.java' and a 'Run' window. The code in 'Employee.java' is as follows:

```

1  package main;
2  import org.slf4j.Logger;
3  import org.slf4j.LoggerFactory;
4
5  public class Employee {
6
7      private static final Logger logger = 5 usages
8          LoggerFactory.getLogger(Employee.class);
9
10     public static void main(String[] args) {
11
12         logger.info("Application Started");
13
14         int salary = 50000;
15
16         logger.debug("Employee Salary : {}", salary);
17
18         if(salary > 40000)
19             logger.warn("High Salary");
20
21         try {
22             int result = 10 / 0;
23         }
24         catch(Exception e){
25             logger.error("Exception Occurred : {}", e.getMessage());
26         }
27         logger.info("Application Ended");
28     }
29 }
30

```

The 'Run' window shows the following output:

```

C:\Users\vagnatharun\.jdk\openjdk-23\bin\java.exe ...
[main] INFO main.Employee - Application Started
[main] WARN main.Employee - High Salary
[main] ERROR main.Employee - Exception Occurred : / by zero
[main] INFO main.Employee - Application Ended
Process finished with exit code 0

```

Explanation

In the above example, the Logger object is created using LoggerFactory. Different log levels are used to display different types of information. The INFO message indicates the start and end of the application, DEBUG displays the employee salary, WARN alerts that the salary is higher than a specified value, and ERROR logs the exception

generated during division by zero. This structured logging approach makes debugging easier than using `System.out.println()` statements.

SLF4J vs `System.out.println()`

SLF4J	SYSTEM.OUT.PRINTLN()
SUPPORTS LOG LEVELS	No log levels
CAN WRITE LOGS TO FILES	Prints only to console
BETTER PERFORMANCE	Less efficient
SUITABLE FOR PRODUCTION	Suitable only for simple programs

Lombok

Project Lombok is a Java library that automatically generates commonly used code such as getters, setters, constructors, `toString()`, `equals()`, and `hashCode()` methods using annotations. By reducing repetitive boilerplate code, Lombok makes Java programs shorter, cleaner, and easier to maintain.

In traditional Java programming, developers spend a significant amount of time writing methods that simply access or modify object data. Lombok eliminates this repetitive work by generating these methods automatically during compilation. As a result, developers can focus more on implementing business logic rather than writing repetitive code.

Lombok is widely used in Spring Boot applications because it improves code readability and reduces file size. Modern IDEs such as IntelliJ IDEA and Eclipse provide built-in support for Lombok through plugins.

Common Lombok Annotations

Annotation	Purpose
@Getter	Generates getter methods.
@Setter	Generates setter methods.
@ToString	Generates <code>toString()</code> method.
@NoArgsConstructor	Generates a no-argument constructor.
@AllArgsConstructor	Generates a constructor with all fields.
@Data	Generates getters, setters, constructors, <code>equals()</code> , <code>hashCode()</code> , and <code>toString()</code> .
@Builder	Implements the Builder Design Pattern.
@Slf4j	Creates an SLF4J logger automatically.

Example:

```
import lombok.Data;

import lombok.AllArgsConstructor;

import lombok.NoArgsConstructor;

@Data
```

@AllArgsConstructor

@NoArgsConstructor

```
public class Student {  
    private int id;  
    private String name;  
}
```

Example using @Builder

```
import lombok.Builder;  
import lombok.ToString;
```

@Builder

@ToString

```
public class Employee {  
    private int id;  
    private String name;  
    private double salary;  
}
```

Main Class

```
public class Main {  
    public static void main(String[] args) {  
        Employee employee = Employee.builder()  
            .id(101)  
            .name("Yagna")  
            .salary(65000)  
            .build();  
        System.out.println(employee);  
    }  
}
```

Output

Employee(id=101, name=Yagna, salary=65000.0)